

```

using System;
using System.Text.Json;
using LearnOpenTK.Common;
using OpenTK.Mathematics;
using OpenTK.Graphics.OpenGL4;
using OpenTK.Windowing.Common;
using OpenTK.Windowing.Desktop;
using OpenTK.Windowing.GraphicsLibraryFramework;

public class Window : GameWindow
{
    public RetrievedMaterial[] ?matData;

    public Dictionary<string, Mesh> AssetCollection {get; set;}

    public string levelFilePath {get; set;}

    private Level _level=new Level();

    public Window(GameWindowSettings gameWindowSettings, NativeWindowSettings nativeWindowSettings)
        : base(gameWindowSettings, nativeWindowSettings)
    {
        _camera=new Camera(Vector3.UnitZ*3,Size.X / (float)Size.Y);
        levelFilePath="assets/level.json";
        AssetCollection=new Dictionary<string,Mesh>();

        // Monitor and resolution
        MonitorInfo minfo = Monitors.GetMonitorFromWindow(this);
        _horizontalResolution=minfo.HorizontalResolution;
        _verticalResolution=minfo.VerticalResolution;

        _controller=new Controller(_horizontalResolution,_verticalResolution);
    }

    protected void UpdateGameState(float deltaTime){

        // Camera
        Vector3 movement=_controller.GetMovement();

        _camera.Position += _camera.Front*movement.X;
        _camera.Position += _camera.Right*movement.Y;
        _camera.Position += _camera.Up*movement.Z;

        Angles2D deltaAngles=_controller.GetArmOrientation();
        Angles2D cameraAngles = new Angles2D(_camera.Yaw,_camera.Pitch);
        cameraAngles += deltaAngles;
        _camera.Yaw = (float)cameraAngles.Yaw;
        _camera.Pitch = (float)cameraAngles.Pitch;
    }

    protected void InitializeLevel()
    {
        _level=new Level(levelFilePath);
        _level.LoadLevel(AssetCollection);
    }

    protected override void OnLoad()

```

```

{
    base.OnLoad();
    InitializeLevel();

    GL.ClearColor(0.2f,0.2f,0.2f,1.0f); // Color de borrado
    GL.Enable(EnableCap.CullFace); // Elimina las caras traseras
    GL.Enable(EnableCap.DepthTest);

    _shader=new Shader("Shaders/shader.vert", "Shaders/shader.frag");
    _shader.Use();

    List<string> activeMeshes = _level.GetActiveMeshes();

foreach(string meshid in activeMeshes){

    if(AssetCollection[meshid] is null )
        throw new Exception("Mesh with empty data");
    if(AssetCollection[meshid].vertexData is null )
        throw new Exception("Mesh with empty data");

    int _vertexBuffer=GL.GenBuffer();
    AssetCollection[meshid].vertexBuffer=_vertexBuffer;
    GL.BindBuffer(BufferTarget.ArrayBuffer,_vertexBuffer);
    GL.BufferData(BufferTarget.ArrayBuffer,
        AssetCollection[meshid].vertexData.Length*sizeof(float),
        AssetCollection[meshid].vertexData,
        BufferUsageHint.StaticDraw);

    int _vertexArray=GL.GenVertexArray();
    AssetCollection[meshid].vertexArray=_vertexArray;
    GL.BindVertexArray(_vertexArray);

    int _indexBuffer=GL.GenBuffer();
    AssetCollection[meshid].indexBuffer=_indexBuffer;
    GL.BindBuffer(BufferTarget.ElementArrayBuffer,_indexBuffer);
    GL.BufferData(BufferTarget.ElementArrayBuffer,
        AssetCollection[meshid].indexData.Length*sizeof(int),
        AssetCollection[meshid].indexData,BufferUsageHint.StaticDraw);

    // Paso 14. Creamos el VAO para el atributo aPosition del shader
    var posLocation = _shader.GetAttribLocation("aPosition");
    GL.EnableVertexAttribArray(posLocation);
    GL.VertexAttribPointer(posLocation,3,VertexAttribPointerType.Float,false,4*
sizeof(float),0);

    // Paso 15. Creamos el VAO para el atributo aWeight del shader
    var colorLocation = _shader.GetAttribLocation("aWeight");
    GL.EnableVertexAttribArray(colorLocation);
    GL.VertexAttribPointer(colorLocation,1,VertexAttribPointerType.Float,false,4
*sizeof(float),3*sizeof(float));

    // Unbind VBO, EBO and VAO
    GL.BindBuffer(BufferTarget.ElementArrayBuffer,0);
    GL.BindBuffer(BufferTarget.ArrayBuffer,0);
    GL.BindVertexArray(0);

}

}

protected override void OnUpdateFrame(FrameEventArgs e)
{
    base.OnUpdateFrame(e);
    if (KeyboardState.IsKeyDown(Keys.Escape))

```

```

    {
        // If it is, close the window.
        Close();
    }
    //Matrix4.CreateFromAxisAngle(_Axis,_RotAngle,out _Model);
    //_RotAngle+=_RotSpeed*(float)e.Time;
    //if(_RotAngle>=MathHelper.TwoPi)
    // Controller Update
    _controller.UpdateState(this.KeyboardState,this.MouseState,e);

    // Update GameState
    UpdateGameState((float)e.Time);
}

protected override void OnRenderFrame(FrameEventArgs args)
{
    // Sin _shader or _mesh no podemos hacer nada
    if(_shader==null)
    {
        return;
    }
    base.OnRenderFrame(args);
    GL.Enable(EnableCap.DepthTest);
    GL.Enable(EnableCap.StencilTest);

    GL.Clear(ClearBufferMask.ColorBufferBit | ClearBufferMask.DepthBufferBit | C
    learBufferMask.StencilBufferBit );
    GL.StencilOp(StencilOp.Keep, StencilOp.Keep, StencilOp.Replace);
    GL.StencilFunc(StencilFunction.Always,1,0xFF);
    GL.StencilMask(0xFF);

    List<string> activeMeshes = _level.GetActiveMeshes();

    foreach(string actorid in _level.ActorCollection.Keys){
        Actor actor=_level.ActorCollection[actorid];
        if( !actor.Enabled)
            continue;
        Mesh ?mesh= AssetCollection[actor.StaticMeshId];
        if(mesh is null)
            throw new Exception("Trying to render an actor without mesh");

        // Binding mesh VAO
        GL.BindVertexArray(mesh.vertexArray);

        _shader.SetMatrix4("model",actor.Model);
        _shader.SetMatrix4("view",_camera.GetViewMatrix());
        _shader.SetMatrix4("projection",_camera.GetProjectionMatrix());

        // Paso 20. Lanzamos la orden Draw
        if (mesh.indexData is not null && mesh.slotData is not null){
            for(int i=0;i<mesh.slotData.Length;i++){
                RetrievedMaterial mat=mesh.matData[i];
                float[] color= mat.diffuse_color;
                Vector3 vcolor=new Vector3(color[0],color[1],color[2]);
                _shader.SetVector3("diffuse_color",vcolor);
                int nelements=0;
                if(i==(mesh.slotData.Length-1))
                    nelements=mesh.indexData.Length-mesh.slotData[i];
                else
                    nelements=mesh.slotData[i+1]-mesh.slotData[i];
                GL.DrawElements(PrimitiveType.Triangles,nelements,DrawElementsTy
                pe.UnsignedInt, ref mesh.indexData[mesh.slotData[i]]);
            }
        }
    }
}

```

```

    }
}
GL.StencilFunc(StencilFunction.Notequal, 1, 0xFF);
GL.StencilMask(0x00);
GL.Disable(EnableCap.DepthTest);
actor.SaveModel();
actor.Scale(new Vector3(1.02f, 1.02f, 1.02f));

_shader.SetMatrix4("model", actor.Model);
if (mesh.indexData is not null && mesh.slotData is not null) {
    for (int i=0; i<mesh.slotData.Length; i++)
    {
        Vector3 vcolor=new Vector3(0.0f, 0.0f, 0.0f);
        _shader.SetVector3("diffuse_color", vcolor);
        int nelements=0;
        if (i==(mesh.slotData.Length-1))
            nelements=mesh.indexData.Length-mesh.slotData[i];
        else
            nelements=mesh.slotData[i+1]-mesh.slotData[i];
        GL.DrawElements(PrimitiveType.Triangles, nelements, DrawElementsType.UnsignedInt, ref mesh.indexData[mesh.slotData[i]]);
    }
}
GL.StencilFunc(StencilFunction.Always, 1, 0xFF);
GL.StencilMask(0xFF);
GL.Enable(EnableCap.DepthTest);
actor.RestoreModel();

GL.BindVertexArray(0);

} // Loop sobre los actores

// Paso 21. Hacemos el swap del doble buffer.
SwapBuffers();

}

protected override void OnResize(ResizeEventArgs e)
{
    base.OnResize(e);
    GL.Viewport(0, 0, Size.X, Size.Y);
}

protected override void OnUnload()
{
    GL.BindBuffer(BufferTarget.ArrayBuffer, 0);
    GL.BindBuffer(BufferTarget.ElementArrayBuffer, 0);
    GL.BindVertexArray(0);

    base.OnUnload();
}

private Shader? _shader ;

private Camera _camera;

private Controller _controller;
private int _horizontalResolution;
private int _verticalResolution;

```

```
}
```